COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 031 - Visualizations and Data Analysis	**SECTION:**	CPE21S1
DATE PERFORMED:	October 17, 2026	**DATE SUBMITTED:**	October 24, 2026
NAME:	Canoy, Hail B. Corro, Yazinel De Leon, Benj Federic	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12 Data Dashboard Taxonomies

1. PROCEDURE OUTPUT

Task 1

[1] ✓ 4s

```python
import pandas as pd
import numpy as np
import plotly.express as px
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Simulated store sales data
np.random.seed(1)
months = pd.date_range("2024-01-01", periods=12, freq="ME")
branches = ["Manila", "Cebu", "Davao", "Baguio"]
categories = ["Food", "Drinks", "Snacks"]

data = {
    "Month": np.random.choice(months, 200),
    "Branch": np.random.choice(branches, 200),
    "Category": np.random.choice(categories, 200),
    "Sales": np.random.randint(5000, 20000, 200),
    "Profit": np.random.randint(500, 5000, 200)
}

df = pd.DataFrame(data)
df.head()
```

```python
# Aggregated views
sales_by_branch = df.groupby("Branch", as_index=False)["Sales"].sum()
sales_by_category = df.groupby("Category", as_index=False)["Sales"].sum()
sales_trend = df.groupby("Month", as_index=False)["Sales"].sum().sort_values("Month")

# Create dashboard layout
fig = make_subplots(rows=2, cols=2,
                    subplot_titles=("Sales by Branch", "Sales by Category", "Monthly Sales Trend"),
                    specs=[[{"type": "bar"}, {"type": "pie"}],
                           [{"colspan": 2}, None]])

# Add charts
fig.add_trace(go.Bar(x=sales_by_branch["Branch"], y=sales_by_branch["Sales"], name="Branch Sales"), row=1, col=1)
fig.add_trace(go.Pie(labels=sales_by_category["Category"], values=sales_by_category["Sales"], name="Category Sales"), row=1, col=2)
fig.add_trace(go.Scatter(x=sales_trend["Month"], y=sales_trend["Sales"], mode="lines+markers", name="Monthly Trend"), row=2, col=1)

# Labels for readability
fig.update_xaxes(title_text="Branch", row=1, col=1)
fig.update_yaxes(title_text="Total Sales (₱)", row=1, col=1)
fig.update_xaxes(title_text="Month", row=2, col=1)
fig.update_yaxes(title_text="Total Sales (₱)", row=2, col=1)

fig.update_layout(height=700, width=950, title_text="🇵🇭 Store Sales Dashboard")
fig.show()
```

Output


Store Sales Dashboard
Sales by Branch
(Manila, 706.711k) Branch Sales
Total Sales (₱)
600k
400k
200k
0
Baguio
Cebu
Davao
Manila
Branch
Sales by Category
34.4%
37.1%
28.5%
Branch Sales
Snacks
Food
Drinks
Monthly Trend
Monthly Sales Trend
Total Sales (₱)
400k
350k
300k
250k
200k
150k
Mar 2024
May 2024
Jul 2024
Sep 2024
Nov 2024
Jan 2025
Month

Task 2

```python
# Interactive feature 1: Dropdown menu to select a branch
fig = go.Figure()

for b in branches:
    branch_data = df[df["Branch"] == b].groupby("Category", as_index=False)["Sales"].sum()
    fig.add_trace(go.Bar(x=branch_data["Category"], y=branch_data["Sales"],
                         name=b, visible=(b == branches[0])))

# Dropdown buttons
buttons = []
for i, b in enumerate(branches):
    visible = [False] * len(branches)
    visible[i] = True
    buttons.append(dict(label=b, method="update",
                        args=[{"visible": visible},
                              {"title": f"Sales by Category - {b} Branch"}]))

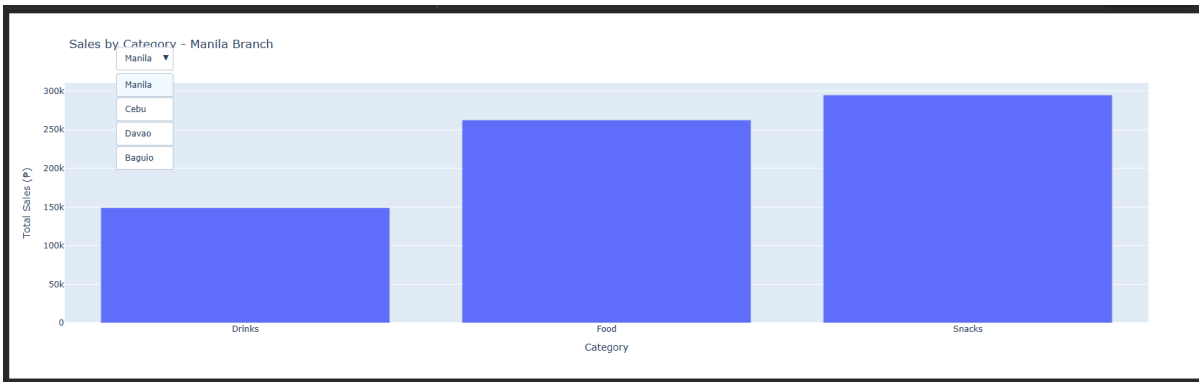
fig.update_layout(
    updatemenus=[dict(buttons=buttons, direction="down", x=0.1, y=1.15)],
    title=f"Sales by Category - {branches[0]} Branch",
    xaxis_title="Category", yaxis_title="Total Sales (₱)"
)
fig.show()
```

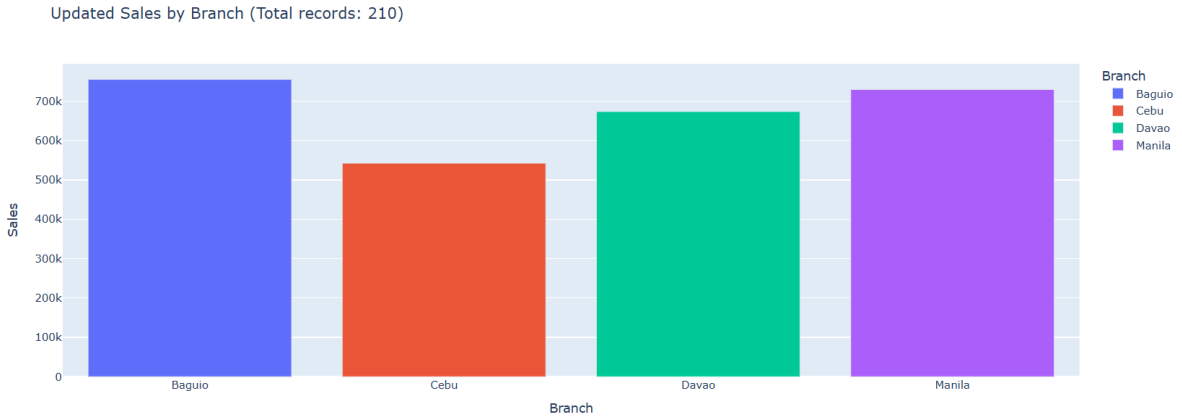
```python
# Interactive feature 2: Dynamic update (simulating real-time data)
def refresh_dashboard(df):
    # Add 10 new random sales records (like new data arriving)
    new_data = pd.DataFrame({
        "Month": np.random.choice(months, 10),
        "Branch": np.random.choice(branches, 10),
        "Category": np.random.choice(categories, 10),
        "Sales": np.random.randint(5000, 20000, 10),
        "Profit": np.random.randint(500, 5000, 10)
    })
    df = pd.concat([df, new_data], ignore_index=True)

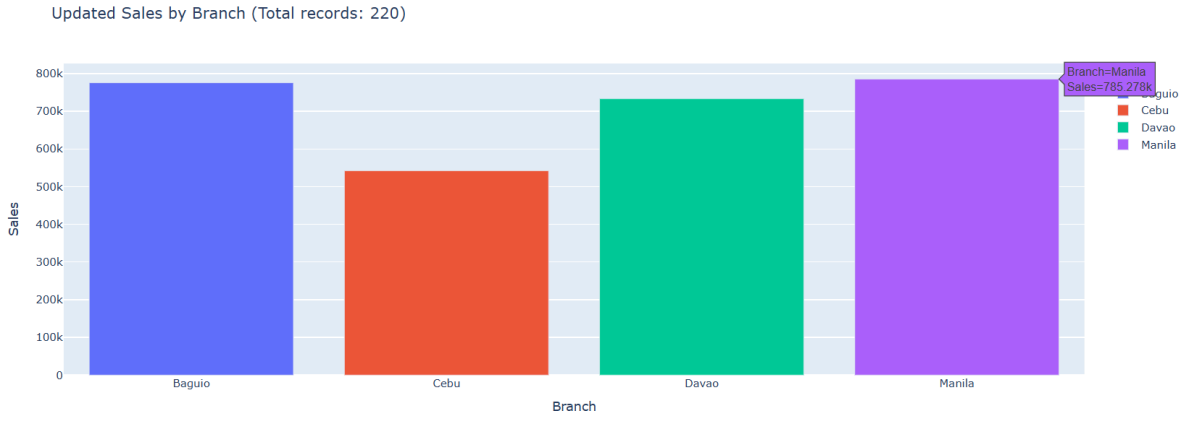
    # Redraw the chart with the updated data
    updated = df.groupby("Branch", as_index=False)["Sales"].sum()
    fig = px.bar(updated, x="Branch", y="Sales", color="Branch",
                 title=f"Updated Sales by Branch (Total records: {len(df)})")
    fig.show()
    return df

# Run the update 2 times
df = refresh_dashboard(df)
df = refresh_dashboard(df)
```

Output







Explanation
Data Update Type: Real-Time (simulated). The refresh_dashboard() function adds new data and redraws the chart, like a live dashboard. A real one would pull from a database or sensor. **User Interaction:** Filter. The dropdown lets the user pick a branch. Hovering over the charts shows exact

values.

Organizational Function: Analytical. It lets managers compare branches and categories to find which ones sell best.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

o

```
#Simulated student data
np.random.seed(5)
weeks = list(range(1, 11))
subjects = ["Math", "Science", "English", "Programming", "Data Analytics"]

df = pd.DataFrame({
    "Week": np.tile(weeks, len(subjects)),
    "Subject": np.repeat(subjects, len(weeks)),
    "Grade": np.random.randint(70, 100, 50),
    "Attendance (%)": np.random.randint(75, 100, 50)
})

#Aggregated data
avg_grades = df.groupby("Subject", as_index=False)["Grade"].mean()
attendance_week = df.groupby("Week", as_index=False)["Attendance (%)"].mean()

#Dashboard layout
fig = make_subplots(rows=2, cols=2,
                    subplot_titles=("Average Grade by Subject", "Attendance by Week", "Performance Trend per
                    specs=[[{"type": "bar"}, {"type": "scatter"}],
                           [{"colspan": 2}, None]])

#Chart 1: Average grade by subject
fig.add_trace(go.Bar(x=avg_grades["Subject"], y=avg_grades["Grade"],
                     name="Average Grade", showlegend=False), row=1, col=1)

#Chart 2: Attendance by week
fig.add_trace(go.Scatter(x=attendance_week["Week"], y=attendance_week["Attendance (%)"],
                         mode="lines+markers", name="Attendance", showlegend=False), row=1, col=2)
```

```
#Chart 3: Performance trend (one line per subject)
for s in subjects:
    sub = df[df["Subject"] == s]
    fig.add_trace(go.Scatter(x=sub["Week"], y=sub["Grade"], mode="lines+markers",
                             name=s, visible=(s == subjects[0])), row=2, col=1)

#Interactive feature: dropdown to choose subject for the trend chart
buttons = []
for i, s in enumerate(subjects):
    visible = [True, True] + [j == i for j in range(len(subjects))]
    buttons.append(dict(label=s, method="update", args=[{"visible": visible}]))

buttons.append(dict(label="All Subjects", method="update",
                    args=[{"visible": [True, True] + [True] * len(subjects)}]))

fig.update_layout(
    updatemenus=[dict(buttons=buttons, direction="down", x=0.1, y=0.42)],
    height=700, width=950, title_text=" Student Grades and Attendance Dashboard"
)
fig.update_xaxes(title_text="Week", row=1, col=2)
fig.update_xaxes(title_text="Week", row=2, col=1)
fig.update_yaxes(title_text="Grade", row=2, col=1)
fig.show()
```

Code



How the dashboard helps an instructor or department:
The bar chart shows which subjects have low average grades, so the department knows where students need more help. The attendance chart shows which weeks had low attendance, so the instructor can look for the cause, such as exam weeks. The trend chart, filtered with the dropdown, shows whether a subject is improving or getting worse. With this, instructors can act early (extra lessons, reminders) based on data instead of guessing.

Conclusion (Canoy)
In this activity, I learned how to combine different charts into one dashboard using Plotly. I also learned how filters and drop-down menus help users view the data they need. This activity helped me understand how dashboards make information easier to read and help people make better decisions.

Conclusion (Corro)
I learned about structuring multi-chart layouts, selecting appropriate chart types for categorical and temporal data, and implementing interactive dropdown filters to customize data views. Additionally, I learned how to categorize dashboards based on update frequency, user interaction, and organizational function to support proactive, data-driven decision-making

Conclusion (De Leon)
Dashboards put many charts in one spot so data is simple and easy to see. Good dashboards use clear colors and the right charts for the info. They can update on their own or let you click filters and drop-down menus to change what you see. They are used to help big groups plan, study facts, or run daily work. Tools like Plotly help make these charts move and update live so people can understand facts quickly and make fast choices.

COLAB LINKS	
Canoy	Hands_On_Activity_12___Data_Dashboard_Ta…
Corro	Hands_On_Activity_12___Data_Dashboard_Ta…
De Leon	Hands_On_Activity_12___Data_Dashboard_Taxonomies-1_DE LEON.ipynb

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.